SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	PRITHIYANKA SHREE M	**Reg. No.:**	192521144
Section / Batch:	2025	**Date:**	08-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

An image compression system performs large matrix transformations to reduce file size. Analyze how divide and conquer matrix multiplication techniques can improve performance. Identify computational challenges. Apply optimized methods such as Strassen's approach. Analyze time complexity and compare with traditional methods. Design a solution and justify its efficiency.

Code of Conduct

I M PRITHIYANKA SHREE (192521144) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: PRITHIYANKA SHREE M

RUBRICS FOR CALCULATION

Case Study					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Your Code

```python
# Divide and Conquer Matrix Multiplication (Strassen's Algorithm)

def add_matrix(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = A[i][j] + B[i][j]
    return C

def sub_matrix(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = A[i][j] - B[i][j]
    return C

def strassen(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0] * B[0][0]]]

    k = n // 2
    # Divide matrices into quadrants
    A11 = [[A[i][j] for j in range(k)] for i in range(k)]
    A12 = [[A[i][j] for j in range(k, n)] for i in range(k)]
    A21 = [[A[i][j] for j in range(k)] for i in range(k, n)]
    A22 = [[A[i][j] for j in range(k, n)] for i in range(k, n)]

    B11 = [[B[i][j] for j in range(k)] for i in range(k)]
    B12 = [[B[i][j] for j in range(k, n)] for i in range(k)]
    B21 = [[B[i][j] for j in range(k)] for i in range(k, n)]
    B22 = [[B[i][j] for j in range(k, n)] for i in range(k, n)]

    # Compute seven products
    M1 = strassen(add_matrix(A11, A22), add_matrix(B11, B22))
    M2 = strassen(add_matrix(A21, A22), B11)
    M3 = strassen(A11, sub_matrix(B12, B22))
    M4 = strassen(A22, sub_matrix(B21, B11))
    M5 = strassen(add_matrix(A11, A12), B22)
    M6 = strassen(sub_matrix(A21, A11), add_matrix(B11, B12))
    M7 = strassen(sub_matrix(A12, A22), add_matrix(B21, B22))

    # Combine results
    C11 = add_matrix(sub_matrix(add_matrix(M1, M4), M5), M7)
    C12 = add_matrix(M3, M5)
    C21 = add_matrix(M2, M4)
    C22 = add_matrix(sub_matrix(add_matrix(M1, M3), M2), M6)

```

```
    # Merge quadrants
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(k):
        for j in range(k):
            C[i][j] = C11[i][j]
            C[i][j + k] = C12[i][j]
            C[i + k][j] = C21[i][j]
            C[i + k][j + k] = C22[i][j]
    return C

# Example matrices
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = strassen(A, B)

print("Resultant Matrix:")
for row in C:
    print(row)
```

INPUT AND OUTPUT: